

OPENUSD GEOSPATIAL · CO-SUBMISSION PREVIEW

usdGeospatial

a codeless CRS schema for OpenUSD, proven by two runtimes

schema declares · runtime resolves · one real-world coordinate frame

A codeless geospatial CRS schema

- Pure-data schema: a CRS prim + a binding API, skipCodeGeneration=true (no compiled types).
- All resolution behavior lives in the example runtimes, not the schema.
- Same shape as the landed Gaussian / particleField contribution: schema + sample runtime(s) + converter + docs.
- Runtimes here: Python `resolve_runtime.py` and a C++ Hydra scene index.

The design call

Resolve at runtime; keep the authored scene coordinate-neutral.

REPLACE · WRAP · OR COEXIST?

Resolve, don't bake

- `crs:binding` is a relationship resolved at runtime — not references-as-binding, not a baked `resetXformStack`.
- The authored scene stays coordinate-neutral; a runtime reprojects and composes ancestor transforms.
- This is the question the earlier effort stalled on — replace, wrap, or coexist with the `UsdGeomXformable` stack? The answer here is **coexist**.
- Proven, not argued: the baked Esri scene and our neutral scene land the same corner to 0.0 mm.

Anchor injection: inject, don't bake

- A georeferenced anchor with an ordinary Cartesian subtree (a building in local metres) — children have no `crs:position`.
- The runtime injects the anchor's rigid local-frame → ECEF basis (full ENU orientation) at resolve time.
- The subtree then composes under it as ordinary `UsdGeomXformable` — nothing baked into the layer.
- `resetXformStack` lives only on a transient, computed Hydra data source, never the authored scene.

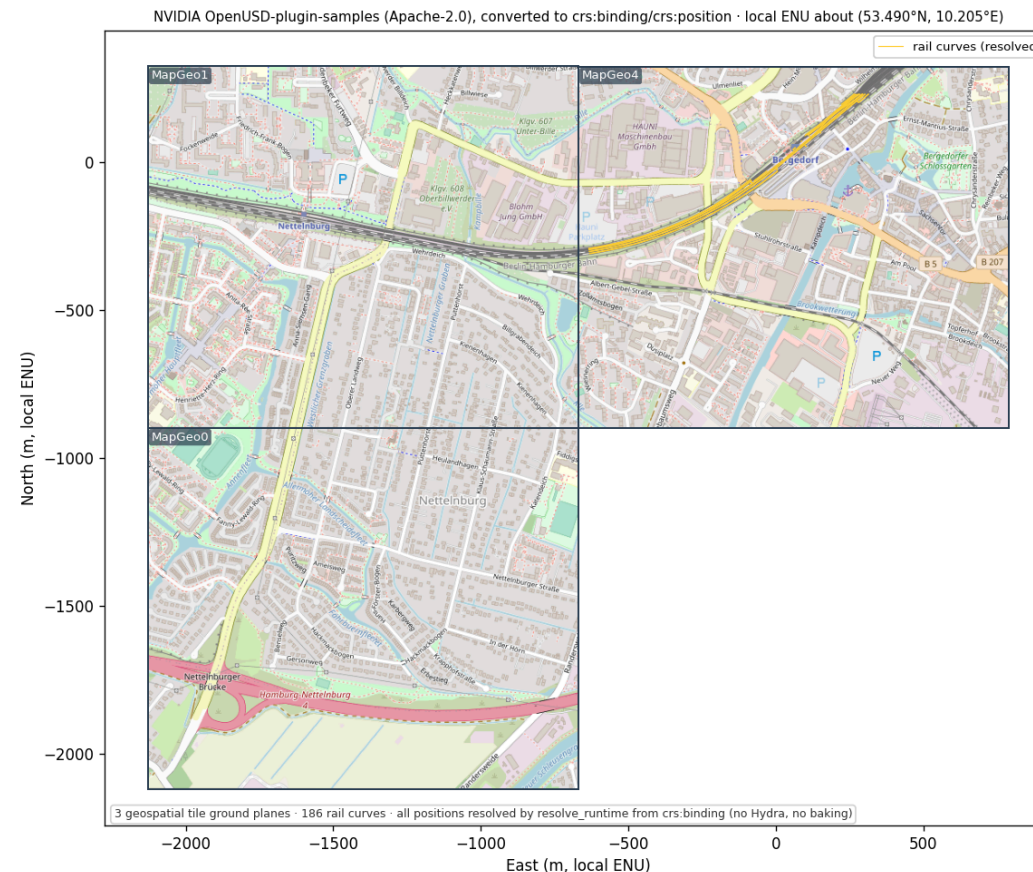
The evidence

Three proofs an independent reviewer can check.

PROOF · NO OVERFIT

Real third-party asset on real tiles

Real third-party asset rendered through the codeless resolver — Deutsche Bahn rails on geospatial tiles

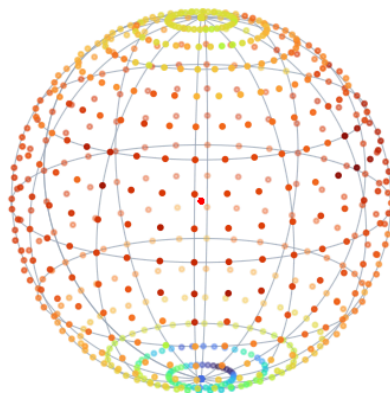


An external Deutsche Bahn railway (~1,470 curves) authored in the original Omniverse schema, converted to crs:binding/crs:position, resolved onto three real geospatial tiles near Hamburg — no Hydra, no baking.

PROOF · CO-REGISTRATION Three CRSs, one ECEF point

3D coherence — schema-resolved geometry co-registers with closed-form WGS84 ground truth (independent of PROJ)

Earth-2 t2m cloud (resolve_runtime) on a closed-form WGS84 graticule — field sits on the right latitudes



red dots @ origin = same cloud resolved
IGNORING crs:binding (negative control)

Cross-CRS co-registration at one benchmark (NOAA NCAT)
three CRSs → one ECEF point → matches closed-form GT

Closed-form WGS84 GT

(first principles, no PROJ)

(1334930.899, -4651062.376, 4141331.101)

SiteGeographic (EPSG:4979)

lon/lat from NOAA NCAT

Δ to GT = 0.000 mm

SiteUTM18N (EPSG:32618)

E/N from NOAA NCAT

Δ to GT = 0.537 mm

SiteNYSPC (EPSG:32118)

State-Plane E/N from NOAA NCAT

Δ to GT = 0.361 mm

three independent CRSs co-register to ≤ 0.537 mm — authored from independent NOAA coords, not by inverting one transform; negative control (ignore binding) lands 6.4×10^6 m off the globe.

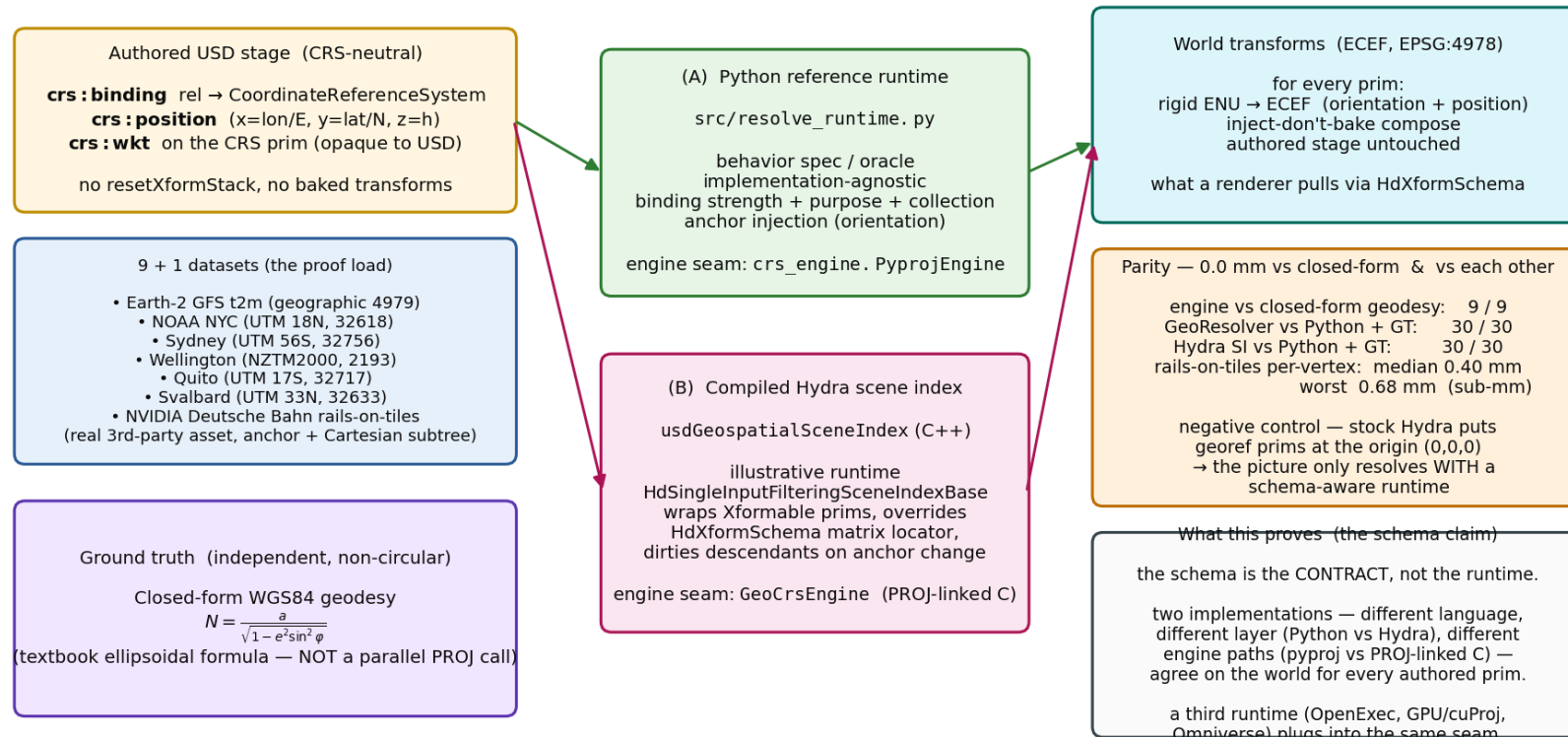
The same monument authored three independent ways (geographic / UTM 18N / NY State Plane) from independent NOAA NCAT coordinates co-registers to one ECEF point to $\leq \sim 0.5$ mm — checked against closed-form WGS84 geodesy, not a parallel PROJ call.

One schema, two runtimes

The schema is the contract; the behavior is plural.

PROOF · CONTRACT NOT IMPLEMENTATION Two independent runtimes, 0.0 mm

One codeless schema, two viable runtime implementations — the schema is the contract; the behavior is plural



The Python reference runtime and the compiled C++ Hydra scene index resolve the same authored stage to the same world, agreeing to 0.0 mm. A third runtime plugs into the same seam.

NO EXACT PRECEDENT — BY DESIGN

Where the Python reference runtime sits

- It's the codeless schema's **executable specification / conformance oracle** — 'given this stage, where does each prim end up?'
- NOT proposed for USD core, NOT a runtime dependency, NOT Python-in-the-render-loop, NOT the prescribed consumer.
- Real consumers (Hydra, OpenExec, Omniverse, GPU cuProj) implement the same contract; the Python is the spec they conform to.
- New part: a runnable reference as the normative behavior for a codeless schema whose behavior is deliberately external.

Open questions

What we'd most like the working group's read on.

Open design questions

- 1. Is 'codeless schema + a runnable reference runtime as the behavior contract' the right shape — and where should that reference ultimately live?
- 2. Is **coexist** the right relationship to `UsdGeomXformable` (neutral scene + runtime reconciliation) — versus hooking CRS resolution into `Xformable` directly?